REFACTORING-CODE-SMELL-TECHNICAL-DEBT

리팩토링·코드스멜·기술부채

연관 기출 · 127-3-2, 135-1-9, 139-3-3

한눈에 이해

코드스멜은 점검 신호이고, 기술부채는 앞으로 추가로 치를 변경 비용이며, 리팩토링은 동작을 보존해 구조를 개선하는 작업입니다.

정의와 핵심 원리

리팩토링은 관찰 가능한 외부 행동을 바꾸지 않고 내부 구조를 작은 단계로 재구성하는 기법입니다. 기술부채는 빠른 선택이나 학습으로 남은 구조상의 부족이 이후 변경에 추가 비용을 만드는 현상을 설명하는 비유입니다.

• 명확하지 않은 요구는 임시 분기·중복·예외 누락을 만들 수 있습니다. 다만 모든 결함과 스멜이 곧 기술부채인 것은 아니고, 기술부채는 후속 작업의 이자 비용과 갚을 원금을 함께 따져 관리해야 합니다.

• 리팩토링 순서는 변경 목적·행동 계약 확인→회귀시험 확보→작은 변환→시험→반복→구조와 문서 정합성 확인입니다. 기능 추가와 구조 변경을 구별하면 실패의 원인을 좁히기 쉽습니다.

• 긴 메서드는 Extract Function으로 책임을 나누고, 중복 코드는 공통 의미가 같을 때 추출합니다. 큰 클래스는 책임에 따라 Extract Class, 기능 편애는 책임 있는 데이터 쪽으로 Move Function, 원시값 집착은 의미 있는 타입 도입을 검토합니다.

• 부채는 요구·설계/아키텍처·코드·테스트·문서/운영 등 발생 위치와 의도·인지 여부로 구분할 수 있습니다. 등록부에 근거·영향·지연비용·상환안·담당자·기한을 적고 변경 빈도와 위험이 큰 부채부터 일정에 반영합니다. 무조건 전면 재작성하는 것은 상환 전략이 아닙니다.

구조·비교·판단

구분	원리·특징	적용·판단
코드스멜	구조 문제 가능성의 징후	정적 지표와 코드 맥락으로 판단
기술부채	미래의 추가 비용·위험	사업상 선택과 우선순위 관리
리팩토링	행동 보존 구조 변환	작은 단계와 회귀 검증 필요

적용 예제

강사 창작 적용 예시: 할인 규칙의 중복 제거

강사 생성 학습 예제입니다. 공식 기준답안이 아닙니다.

조건: 여러 화면이 서로 다른 할인 조건을 구현합니다.

1. 현행 입출력과 예외를 특성화 시험으로 고정합니다.
2. 한 가지 의미의 할인 계산을 별도 함수/정책 객체로 옮기고 호출자를 하나씩 전환합니다.
3. 불명확했던 규칙은 별도 요구 변경으로 승인하고 시험을 추가합니다.

결과·해석: 구조 정리와 요구 수정의 효과를 구분하여 검증할 수 있습니다.

전제·주의: 의미가 다른 코드를 모양이 비슷하다는 이유만으로 합치면 잘못된 추상화를 만들 수 있습니다.

답안 골격과 혼동 방지

세 개념의 정의·연결 → 코드스멜 3종 이상과 개선기법 → 부채 유형·관리·요구 불명확성 사례

- 행동이 달라졌다면 그 변경 전체를 리팩토링이라고 부르지 않습니다.
- 부채 규모를 LOC나 스멜 개수만으로 확정하지 않습니다.

내용을 가리고 회상

1. Extract Function이 적절하지 않은 중복은 무엇입니까?
2. 부채 원금과 이자는 각각 무엇입니까?
3. 요구 변경과 리팩토링을 어떻게 나누어 검증합니까?

적용 기준과 근거

고전 개념의 설명이며 2026-11-05 열람 원문을 기준으로 했습니다.

내용 검토 상태: 출처 확인. 출처 확인은 공식 정답이나 학습 성취를 뜻하지 않습니다.

출처별 상세 열람 범위는 위키의 출처 기록에서 확인합니다.

- Refactoring - Martin Fowler; 지속 갱신 문서; 확인 2026-11-05.
- Technical Debt - Martin Fowler; 지속 갱신 문서; 확인 2026-11-05.
- Guide to the SWEBOK v4.0a - IEEE Computer Society / Hironori Washizaki 편; 2026-08; 확인 2026-11-05.